

Assignment: 03

Name: SHUVASHIS BASAK

ID: 22301767

Section: 23

Course: CSE340

Answer to the question no. 1

0x ABB26D

$\Rightarrow (1010\ 1011\ 1011\ 1001\ 0110\ 0000\ 1001)_2$

Sign bit = 1

biased exponent (6 bits) = $(010101)_2 = 21$

Bias = $2^{6-1} - 1 = 31$

exponent = $21 - 31 = -10$

Fraction = $11011\ 1001\ 0110\ 0000\ 1001$

$= (0.11011100101100001001)_2$

$= (0.86206)_{10}$

Decimal value = $(-1)^1 \times (1 + 0.86206) \times 2^{-10}$

$= -1.86206 \times 2^{-10}$

$= -1.818417267 \times 10^{-3}$

Answer to the question no. 2

a

$$50.7869 + 72.83 - 22.58$$

$$= 110010.11001 + 100111.11010 - 11101.10010$$

$$= 1.10010110010 \times 2^5 + 1.0011111010 \times 2^6 -$$

$$1.110110010 \times 2^4$$

$$= 0.110010110010 \times 2^6 + 1.0011111010 \times 2^6 - 0.0110110010 \times 2^6$$

$$= (1.000001010011 - 0.0110110010) \times 2^6$$

$$= 1.10010100001 \times 2^6$$

b

$$64.2486 \times 42.1832$$

$$= 1000000.00111 \times 110001.00101$$

$$= 1.00000000111 \times 2^6 \times 1.1000100101 \times 2^5$$

$$= 1.10001010100110000001 \times 2^{11}$$

Answer to the question no. 3

$$\begin{aligned} & 28.4810 - (-4.0210) \\ &= 28.4810 + 4.0210 \\ &= 11100.0111 + 100.00000 \\ &= 1.11000111 \times 2^4 + 1.00000000 \times 2^2 \\ &= 1.11000111 \times 2^4 + 0.010000000 \times 2^4 \\ &= 10.00000111 \times 2^4 \\ &= 1.000000111 \times 2^5 \end{aligned}$$

Now,

$$\text{Bias} = 2^{n-1} - 1 = 2^{8-1} - 1 = 127$$

$$\text{Bias exponent} = 5 + 127 = 132$$

$$\begin{aligned} \text{Range} &= 0 \text{ to } 2^n - 1 \\ &= 0 \text{ to } 255 \end{aligned}$$

$$\text{valid range} = 1 \text{ to } 254$$

As $1 \leq 132 \leq 254$, so, it's not overflow or underflow.

Answer to the question no. 4

a

In IEEE-754 floating point representation, bias is added to map the exponent range to assigned integer values. In this case, negative exponents are encoded as positive integers, which is smaller than the bias, and the positive exponents are encoded as integers larger than the bias.

b

optimized multiplication uses 64-bit ALU instead of the 128-bit ALU used in the first step of long multiplication. It also eliminates the need for a separate multiplier register by storing the multiplier in the lower half of the product register. It reduces the number of registers required. The operation shifts the product register right and adds the multiplied based on the LSB of the product register, streamlining the control logic.

Answer to the question no. 5.

mul:

multiplier $rs1$ and $rs2$, stores the lower 64 bits of the 128-bit product in rd .

syntax: $mul\ rd, rs1, rs2$

example: $mul\ x5, x6, x7$

computes $x6$ multiplier $x7$ and stores lower bits in $x5$.

mulh:

Multiplier $rs1$ and $rs2$ (treated as signed), stores the upper 64 bits of the 128 bit product.

syntax: $mulh\ rd, rs1, rs2$

example: $mulh\ x5, x6, x7$

mulhu
multiplies $rs1$ and $rs2$ (treated as unsigned),
stores the upper 64 bits.

syntax: $mulhu\ rd, rs1, rs2$

example: $mulhu\ x5, x6, x7$

mulhsu

multiplies signed $rs1$ by unsigned $rs2$, stores the
upper 64 bits.

syntax: $mulhsu\ rd, rs1, rs2$

example: $mulhsu\ x5, x6, x7$

Answer to the question no. 6

- ① The comparison instruction (fbc) write a
boolean result into a general-purpose integer
register (rd), not a floating-point register.

(ii) False, The suffix "d" in fld indicates double precision.

(iii) The instruction fld checks if $f_3 \leq f_5$. If x_5 contains 0 (False), it means the condition $f_3 \leq f_5$ is not met. Therefore f_3 is greater than f_5 .

Answer to the question no. 7

seq.d x_5, f_1, f_2

bne $x_5, x_0, \text{jumpEqual}$

beg $x_5, x_0, \text{jumpNotEqual}$

jumpEqual;

jumpNotEqual:

Answer to the question no. 8

flt.d x_5, f_2, f_1

bne $x_5, x_0, \text{jumpGreater}$

srli $x_{19}, x_{19}, 4$

beg $x_5, x_0, \text{jumpNotGreater}$

jumpGreater:

jumpNotGreater:

Answer to the question no. 9

flt.d x_5, f_5, f_4

beg $x_5, x_0, \text{else-label}$

fmadd.d f_6, f_4, f_5, f_7

beg x_0, x_0, Exit

else-label:

fmadd.d f_6, f_4, f_5, f_7

Exit: